

Teaching Pacman to search



Five search algorithms, two multi-goal problems, a CSV trace for every run and one deliberately deceptive maze.
Jaweria Abdul Hameed (24i-3025) • Mohsin Khan (24i-3135)

26/25

autograder points, 25 required
points + 1 bonus point from
the supplied q7 grader

255

nodes expanded by A* on
trickySearch (top threshold:
7000)

47 vs 37

path length, GBFS vs A* on the
custom maze

52

CSV traces in evidence/

What is in this report

- **Section 1** describes how each algorithm is built and what it costs in time and space.
- **Section 2** defines the state spaces for CornersProblem and FoodSearchProblem.
- **Section 3** proves that cornersHeuristic and foodHeuristic are admissible and consistent.
- **Section 4** compares all algorithms empirically. Every number comes from the CSV traces or the run output.
- **Section 5** analyses the custom maze and why greedy search falls for it.
- **Appendix** documents the two places where the PDF and the starter code disagree.

Design rules followed

Only search.py, searchAgents.py and the new layout were edited. Every algorithm is written inside its own provided function, with no helper delegation, and expands successors in the order getSuccessors returns them. The CSV logger is one shared class in search.py, called from inside each loop.

How the CSV trace reads. One row is written for every expanded state (a state whose successors were generated) with the columns iteration, expanded_state, parent, action, generated_successors, frontier_before, frontier_after, explored, g, h, f. The frontier is printed with the next state to be expanded first. DFS, BFS and UCS log $h = 0$ and $f = g$; GBFS logs $f = h$ because it orders by h alone. The goal that ends a search is selected, not expanded, so it has no row: the row count equals the run's *Search nodes expanded*.

```
evidence/astar_24i3025Search_20260926_205918.csv
iteration,expanded_state,parent,action,generated_successors,frontier_before,frontier_after,explored,g,h,f
1,(23, 6),,,[(23, 7) via North, (22, 6) via West],[(23, 6):27],[(22, 6):27, (23, 7):29],[],0,27,27
2,(22, 6),(23, 6),West,[(21, 6) via West],[(22, 6):27, (23, 7):29],[(21, 6):27, (23, 7):29],[(23, 6)],1,26,27
3,(21, 6),(22, 6),West,[(20, 6) via West],[(21, 6):27, (23, 7):29],[(20, 6):27, (23, 7):29],[(23, 6), (22, 6)],2,25,27
```

1 • Algorithms and complexity

All five searches are graph searches: a state is never expanded twice unless A* finds a strictly cheaper path to it. Notation: b branching factor, d depth of the shallowest goal, m maximum depth, C^* optimal cost, ϵ smallest step cost, $|S|$ number of reachable states (free cells in a maze).

ALGORITHM	FRINGE (UTIL.PY)	ORDER BY	TIME	SPACE	COMPLETE	OPTIMAL
DFS	Stack (LIFO)	most recent	$O(b^m)$	$O(\min(S , b^m))$	yes (finite, explored set)	no
BFS	Queue (FIFO)	depth	$O(b^d)$	$O(b^d)$	yes	unit costs
UCS	PriorityQueue	$g(n)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	yes	yes
GBFS	PriorityQueue	$h(n)$	$O(b^m)$	$O(b^m)$	yes (finite, seen set)	no
A*	PriorityQueue	$g(n) + h(n)$	$O(b^d)$ good h	$O(b^d)$	yes	h admissible

In a grid maze every bound is also capped by the number of cells, so DFS and BFS run in $O(|S|)$. Heap push and pop cost $O(\log |S|)$, but the supplied `util.PriorityQueue.update` scans the heap for the item and re-heapifies, which is $O(|F|)$ for a frontier of size $|F| \leq |S|$. UCS and A* therefore cost $O(|E| \cdot |S|)$ in the worst case with this queue; an indexed heap would give the textbook $O(|E| \log |S|)$. Each fringe entry stores its action list, so a run also pays $O(|S| \cdot L)$ memory for paths of length L . That is the price of keeping every algorithm inside one small function and it is negligible at maze sizes.

Implementation notes

- **DFS** marks a state explored when it is popped, skips stale duplicates, and pushes successors in `getSuccessors` order (N, S, E, W), so the LIFO stack expands West first.
- **BFS** checks a *seen* set before pushing, so a state enters the queue once. That keeps the first path found the shortest.
- **UCS** queues the bare state and calls `PriorityQueue.update`, so a cheaper path to a state already in the fringe lowers its priority instead of adding a duplicate. Costs and paths live in dictionaries.
- **GBFS** orders by h only. A state's priority never changes, so it is queued once. The heuristic is passed as `heuristic=name` on the command line through the unchanged `SearchAgent`.
- **A*** uses $f = g + h$ and re-opens a state whenever a cheaper g appears, which keeps it optimal even for an admissible heuristic that is not consistent.



Nodes expanded on bigMaze (path cost 210 for every optimal algorithm). DFS also returns cost 210 here by luck of the maze, not by guarantee.

2 • State spaces

CornersProblem

A state is (position, visitedCorners). position is Pacman's (x, y). visitedCorners is a tuple of the corners reached so far, always kept in the fixed order of self.corners. Two paths that reach the same corners in a different order therefore produce the *same* state, which is what lets graph search merge them.

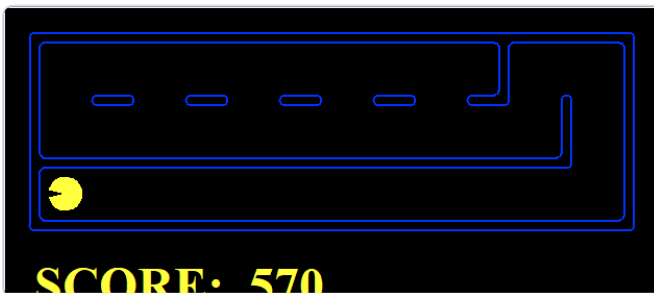
- **Start:** the starting position, with its own cell counted if it is a corner.
- **Goal:** all four corners visited.
- **Successors:** the four moves that do not hit a wall, each with cost 1. Stepping onto a corner adds it to the tuple.
- **Size:** at most $|\text{free cells}| \times 2^4 = 16 \cdot |\text{free cells}|$ states.

FoodSearchProblem

A state is (position, foodGrid), where foodGrid is a Grid of booleans. The goal is an empty grid. The class ships fully implemented and is not edited. The state space is up to $|\text{free cells}| \times 2^{|\text{dots}|}$, which is why an informed heuristic matters: blind search on trickySearch is hopeless, while A* with our heuristic expands only 255 nodes.

AnyFoodSearchProblem and ClosestDotSearchAgent

The goal test is one line: self.food[x][y]. findPathToClosestDot runs BFS, so it returns the nearest dot, and the agent repeats this until the board is empty. It is greedy across dots: on bigSearch it collects everything with a path of 350 steps. It does not promise the shortest tour, only the shortest hop each time.



A* with foodHeuristic on trickySearch: 255 nodes, cost 60.



A* with cornersHeuristic on mediumCorners: 741 nodes, cost 106.

3 • Heuristics and proofs

cornersHeuristic

Remove the walls and count Manhattan distance. The cheapest way to visit the remaining corners in that relaxed maze is the shortest Manhattan tour from Pacman through every unvisited corner. With at most four corners there are at most 24 orders to try, so it is computed exactly on every call.

ADMISSIBLE

Any real path visits the remaining corners in some order. Each leg between two consecutive stops is at least their Manhattan distance, because walls only lengthen a path. So the real cost is at least the length of that order in the relaxed problem, which is at least the best order, which is h . At a goal there are no corners left and $h = 0$.

CONSISTENT

Take any step from s at position p to s' at position p' , with $d(p, p') = 1$. *Case 1, no new corner.* The remaining set is unchanged. Walking from p to p' and then following the optimal tour of s' is one candidate tour from p , so $h(s) \leq d(p, p') + h(s') = 1 + h(s')$. *Case 2, p' is a new corner c .* The tour of s can start with the leg $p \rightarrow c$ of length 1 and then follow the optimal tour of s' over the other corners, so the same inequality holds. Hence $h(s) \leq c(s, a, s') + h(s')$.

foodHeuristic

Let F be the remaining dots and d the true maze distance, computed with the provided `mazeDistance` (BFS) and cached in `problem.heuristicInfo`. The heuristic is

$$h(s) = \min_{f \in F} d(p, f) + \text{MST}_d(F)$$

where $\text{MST}_d(F)$ is the weight of a minimum spanning tree over the dots with maze distances as edge weights (Prim). The tree depends only on which dots are left, so it is cached per dot set. Cost: $O(k^2)$ for one tree of k dots, each pair distance is one BFS the first time it is needed, and $O(k)$ per call afterwards.

ADMISSIBLE

Any solution first reaches some dot f_1 , which costs at least $\min d(p, f)$. From there it is a walk that touches every dot. The hops between consecutive dots form a connected graph on F whose edges each cost at least the maze distance, so the walk costs at least the MST weight. The two parts add.

CONSISTENT

Step from s to s' , cost 1. *Case 1, no dot eaten.* F is unchanged, so the MST term is equal, and $d(\cdot, f)$ changes by at most 1 per step, so the minimum changes by at most 1: $h(s) \leq 1 + h(s')$. *Case 2, dot f eaten,* so $p' = f$ and $F' = F - \{f\}$. If F' is empty, $h(s) = d(p, f) = 1$ and $h(s') = 0$. Otherwise let g be the dot of F' nearest to f . Adding the edge $f-g$ to a minimum tree of F' spans F , so $\text{MST}(F) \leq \text{MST}(F') + d(f, g)$. Also $\min d(p, \cdot) \leq d(p, f) = 1$. Adding, $h(s) \leq 1 + \text{MST}(F') + d(f, g) = 1 + h(s')$.

The supplied heuristic tests check each fixture's start state against the true cost and check its immediate successor edges for a consistency drop; the proofs above establish the general result. On `trickySearch` it reports **255 expanded nodes** against the thresholds 15000 / 12000 / 9000 / 7000, so the question scores 5 of 4.

4 • Empirical results

Every run was made through `pacman.py`, which also wrote its CSV trace. Time is the search only, measured by the agent. The corner and food runs use A* with the heuristics of Section 3.

RUN	PATH COST	NODES EXPANDED	SECONDS
DFS tinyMaze	10	15	0.00
DFS mediumMaze	130	146	0.00
DFS bigMaze	210	390	0.00
BFS mediumMaze	68	269	0.00
BFS bigMaze	210	620	0.10
UCS mediumMaze	68	269	0.00
UCS mediumDenselyMaze (added layout)	54	173	0.00
UCS stayEastSearch (added layout)	23	54	0.00
UCS StayEastSearchAgent mediumMaze	1	260	0.00
GBFS bigMaze (Manhattan)	210	466	0.00
GBFS bigMaze (Euclidean)	210	471	0.10
A* bigMaze (null heuristic)	210	620	0.10
A* bigMaze (Manhattan)	210	549	0.10
BFS CornersProblem tinyCorners	28	252	0.00
A* corners mediumCorners	106	741	0.30
A* food trickySearch	60	255	0.20
ClosestDot bigSearch	350	n/a	n/a

What the numbers say

- BFS and UCS agree on every unit-cost maze (68 on mediumMaze, 210 on bigMaze). UCS pays for the heap and gains nothing until step costs differ.
- On StayEastSearchAgent the cost of a step falls to 0.5^x toward the east. UCS finds a path of total cost 1 while BFS would ignore costs entirely.
- A* with the Manhattan heuristic expands 549 nodes on bigMaze against 620 for the null heuristic, which is UCS in disguise. Both return cost 210, so the heuristic saved work without losing optimality.
- GBFS expands fewer nodes than A* on bigMaze (466 vs 549) and still returns 210, but that is an accident of this maze. Section 5 shows the case where it does not.
- DFS returns 130 on mediumMaze where the optimum is 68: cheap to run, no guarantee on quality.

Nodes expanded, multi-goal problems



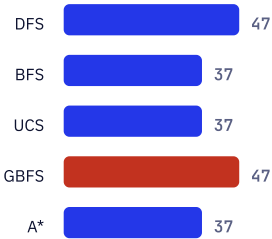
5 • Custom maze: the greedy trap

The layout 24i3025Search.lay is 26×13 cells. Pacman starts at the top right and the food is in the bottom-left corner (1, 1), where SearchAgent expects the goal. It has decision branches, five dead ends and two ways to reach the food.

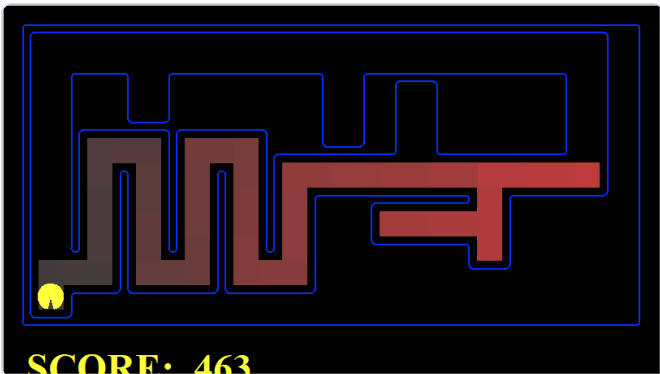
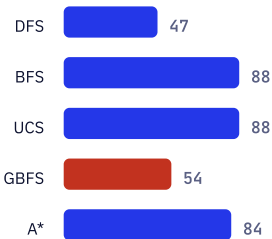
- **Decoy route.** Straight from the start, a corridor runs west, then turns into a long serpentine that hugs the goal. Every cell of it is close to (1, 1) in Manhattan terms, so h is small everywhere in it. It is 47 steps long.
- **True route.** The other way first walks along the top edge, away from the goal, so h rises before it falls, and then down the west column. It is 37 steps long.
- **Dead ends.** Stubs off both corridors and a side pocket give the searches places to waste time.

Greedy search always takes the frontier state with the smallest h . The decoy corridor keeps offering smaller values than the top edge, so GBFS commits to the serpentine and never looks at the top route. A* adds the cost already paid, g . Inside the serpentine g grows quickly while h barely moves, so f climbs above the top route's and A* switches to it.

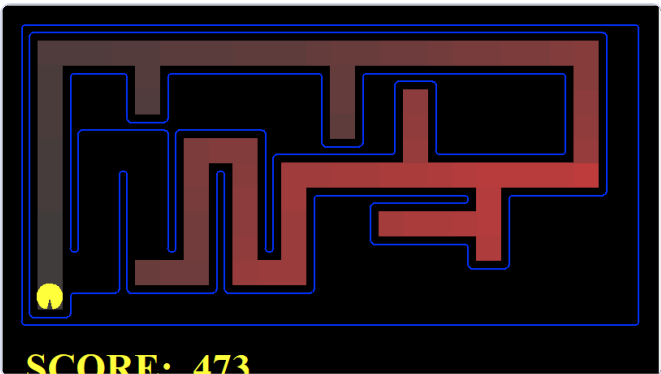
Path cost



Nodes expanded



GBFS: follows the serpentine, cost 47, 54 nodes.



A*: takes the top edge, cost 37, 84 nodes.

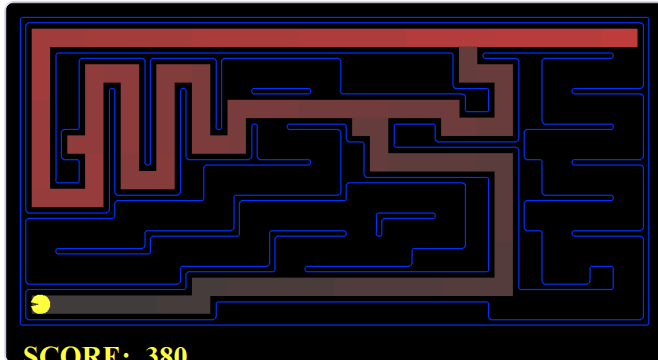
GBFS expands 54 nodes and A* 84, so greedy is cheaper to run, but its path is 10 steps (27%) longer. BFS and UCS find the same optimum as A* with 88 nodes. DFS is not optimal here: it finds a path of cost 47 (same as GBFS) while expanding only 47 nodes.

RUN	PATH COST	NODES EXPANDED	SECONDS
DFS custom maze	47	47	0.00
BFS custom maze	37	88	0.00
UCS custom maze	37	88	0.00
GBFS custom maze	47	54	0.00
A* custom maze	37	84	0.00

Appendix • where the PDF and the starter code disagree

A. North-East-South-West order (Inconsistency #3)

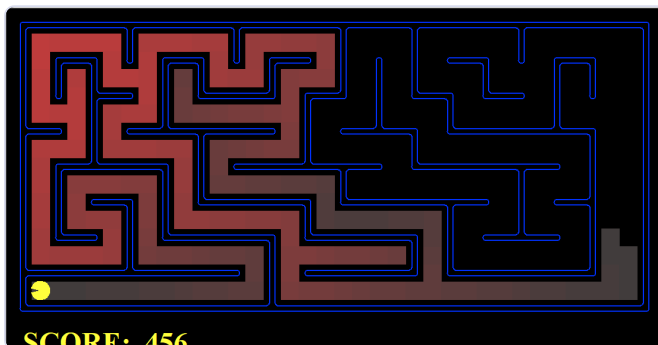
Task 1 names N–E–S–W expansion, but the protected `PositionSearchProblem.getSuccessors` returns N, S, E, W and the autograder q1 reference solutions are generated from that order. Following the teacher instruction that all tests must pass, `depthFirstSearch` pushes successors in the order `getSuccessors` returns them, with no reordering. No other algorithm reorders successors, and no protected file or fixture was changed.



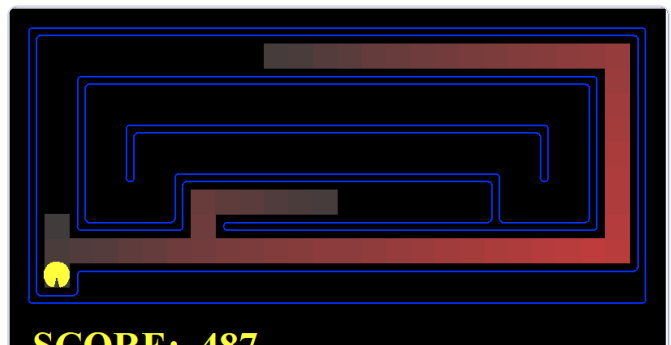
`fn=dfs (getSuccessors order): cost 130, 146 nodes.`

B. Two UCS layouts added (Inconsistency #4)

The PDF lists `-l mediumDenselyMaze` and `-l stayEastSearch`, but neither layout ships with the starter code, so both were created in `layouts/` and the literal commands now run. `mediumDenselyMaze` is a dense corridor maze (UCS cost 54); `stayEastSearch` is a two-route maze (cost 23). Directional cost is still an *agent*, so `-l mediumMaze -p StayEastSearchAgent` is also kept (cost 1).



`mediumDenselyMaze, fn=ucs.`



`stayEastSearch, fn=ucs, cost 23.`

C. Smaller notes

- **CSV logger vs the forbidden files.** The logger needs no change to `pacman.py` or `util.py`; it lives in `search.py`.
- **No GBFS stub or test.** `gbfs` was added to `search.py`; `SearchAgent` finds it with `getattr`.
- **h and f for uninformed searches.** DFS, BFS and UCS log $h = 0$, $f = g$ so every CSV has the same columns.
- **Two-person group.** The layout is named after the first ID, `24i3025Search.lay`.